

SensioLabs

Building an AI Agent with Symfony



SymfonyLive
PARIS 2026

Workshop Objectives

- Build a **ChatBox** UI with Symfony UX Live Components
- Connect it to an **AI Agent** powered by Claude
- Create **Tools** the agent can use to query data
- Enable **Semantic Search** with vector stores

The ChatBox Is a LiveComponent Backed by a Symfony Form

- `#[AsLiveComponent]` — PHP class becomes a reactive UI component
- `#[LiveProp]` / `#[LiveAction]` — state and actions synced with the browser
- `ComponentWithFormTrait` — form submission without a full page reload
- The `ChatBox` UI is already wired in the starter project

```
1  #[AsLiveComponent]
2  class ChatBox extends AbstractController
3  {
4      use DefaultActionTrait;
5      use ComponentWithFormTrait;
6
7      #[LiveProp]
8      public bool $isOpen = false;
9
10     #[LiveAction]
11     public function toggle(): void
12     {
13         $this->isOpen = !$this->isOpen;
14     }
15 }
```

Turbo Streams and Mercure Push Entity Changes to the Browser

- `#[Broadcast]` on a Doctrine entity — auto-publishes changes via Mercure
- `turbo_stream_listen()` in Twig — subscribes the UI to those updates
- The post detail view updates in real time without a page reload
- Mercure Hub is already configured in the starter project

```
1 #[Broadcast]
2 #[ORM\Entity]
3 class Post { /* ... */ }
```

```
1 <div id="post_{{ post.id }}"
2     {{ turbo_stream_listen(post) }}>
3     <twig:Admin:SinglePost :post="post" />
4 </div>
```

Phase 2

Connecting to the AI Agent

AI Agent Configuration

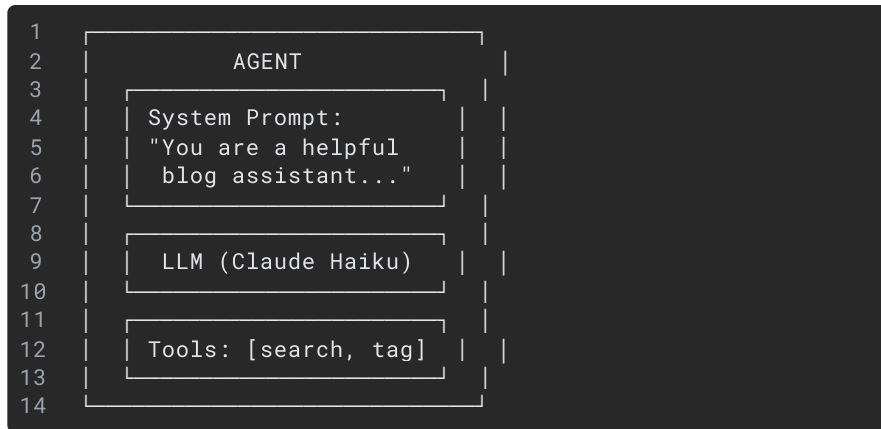
The Symfony AI Bundle provides a unified abstraction for LLM integration.

LLMs Are Stateless Text Predictors

- Predict the most likely next token — no real "understanding"
- **Stateless**: full conversation history must be sent with every request
- Context window is finite — long conversations must be truncated or summarised
- Memory is faked by replaying history; the message store handles persistence

An Agent Wraps the LLM with Instructions

- **System prompt:** personality + rules
- **Platform:** API connection
- **Model:** which LLM to use
- **Tools:** actions it can perform



The Agent Configuration Defines the AI Brain

- **Platform:** API connection (Anthropic)
- **Model:** Which LLM (Claude Haiku)
- **Prompt:** System instructions
- All defined in YAML

```
1 # config/packages/ai.yaml
2 ai:
3   agent:
4     default:
5       platform: 'ai.platform.anthropic'
6       model: 'claude-haiku-4-5-20251001'
7       prompt: |
8         You are a helpful assistant
9         for a blog application.
```

Message Store Persists Conversation History

- Stores messages per conversation
- Enables conversation context
- Backed by Symfony's cache component
- No database table or migration needed

```
ai:  
  message_store:  
    cache:  
      messages:  
        service: cache.app
```

Chat Configuration Links Agent and Store

- Named chat configurations
- References agent by service ID
- References message store
- Injectable as `ChatInterface`

```
ai:  
  chat:  
    app_chat:  
      agent: 'ai.agent.default'  
      message_store: 'ai.message_store.cache.messages'
```

ChatInterface Submits Messages to the Agent

- Inject via constructor
- Create `Message::ofUser()`
- Call `$chat->submit()`
- Response stored automatically

```
1 public function __construct(  
2     private readonly ChatInterface $chat,  
3     private readonly MessageStoreInterface $store,  
4 ) {}  
5  
6 #[LiveAction]  
7 public function save(): void  
8 {  
9     $message = Message::ofUser($content);  
10    $this->chat->submit($message);  
11 }
```

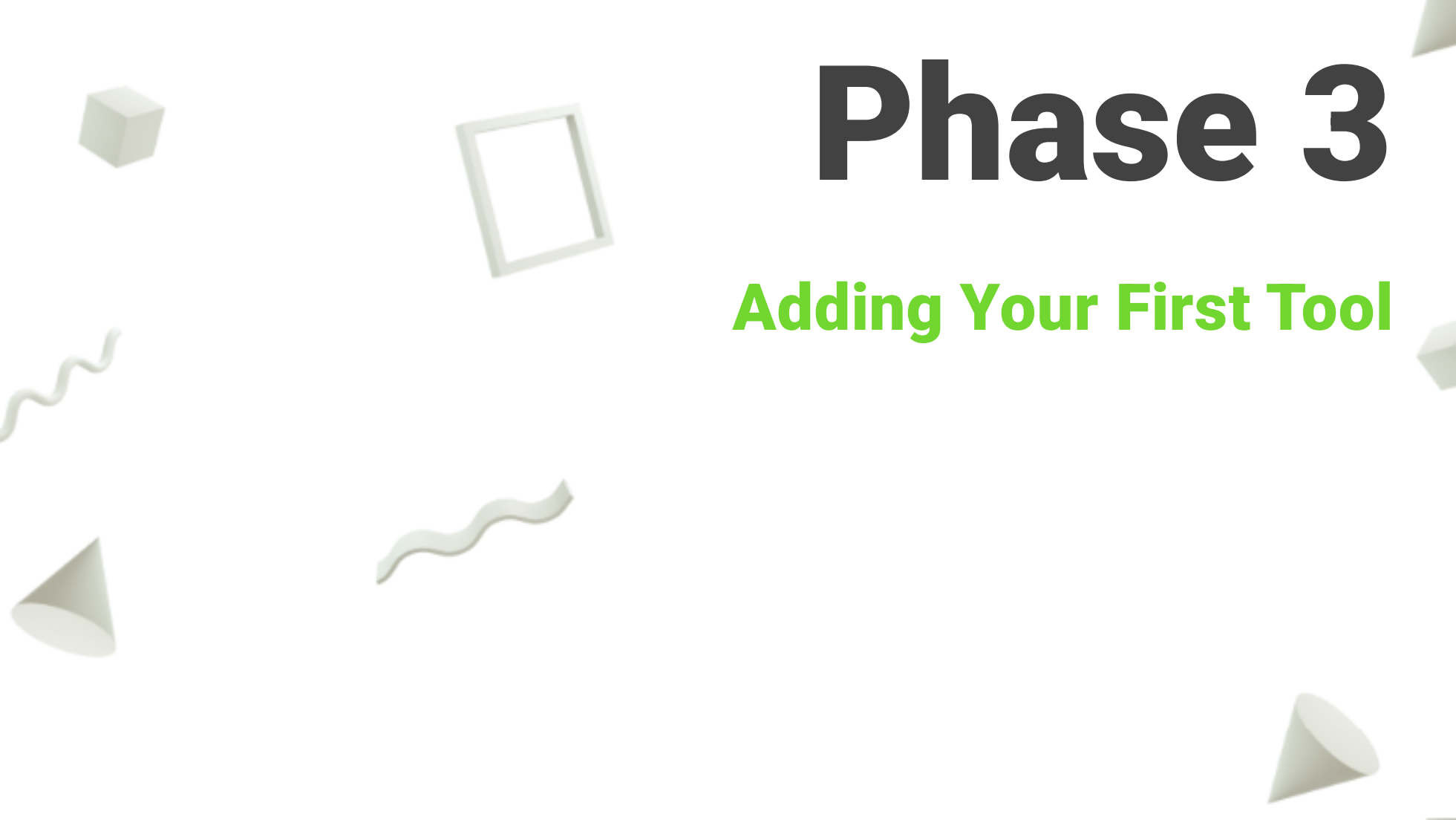
MessageStoreInterface Provides History Access

- `load()` returns MessageBag
- `getMessages()` for array
- Display in template
- Enables context awareness

```
1  #[ExposeInTemplate]
2  public function getMessages(): array
3  {
4      return $this->store
5          ->load()
6          ->getMessages();
7  }
```

Phase 3

Adding Your First Tool



AI Tools

Tools allow the agent to perform actions and query your application data.

Tools Give the LLM "Superpowers"

- LLM reads tool descriptions
- Decides which tool fits the query
- Generates parameters (JSON)
- Your code executes the action

```
1 User: "How many published posts?"
2   ↓
3   [ LLM thinks: "I should use status_search
4     tool with status='published'" ]
5   ↓
6   { "tool": "status_search",
7     "params": {"status": "published"} }
8   ↓
9   [ Your PHP Code Executes
10    → Returns "Found 42 posts" ]
```



This is the **Think** → **Act** → **Observe** loop (ReAct): the LLM reasons about the query, calls a tool, observes the result, then reasons again — repeating until it can answer.

The #AsTool Attribute Registers Agent Tools

- `name`: Identifier for the LLM
- `description`: When to use it
- Class is auto-discovered
- Autowiring works normally

```
1  #[AsTool(  
2      name: 'status_search',  
3      description: 'Counts posts by status',  
4  )]  
5  final class StatusSearch  
6  {  
7      public function __construct(  
8          private readonly PostRepository $repo,  
9      ) {}  
10 }
```

The `__invoke` Method Executes the Tool Logic

- Parameters from LLM request
- Return string result
- LLM receives the response
- Decides next action

```
1 public function __invoke(string $status): string
2 {
3     $count = $this->repo->count([
4         'status' => PostStatus::from($status)
5     ]);
6
7     return "Found {$count} posts";
8 }
```

Tools Must Be Registered in Agent Configuration

- Add FQCN to `tools` array
- Agent discovers capabilities
- LLM decides when to call
- Multiple tools allowed

```
1  ai:
2    agent:
3      default:
4        # ... platform, model, prompt
5      tools:
6        - 'App\Ai\Tool\StatusSearch'
```

Phase 4

Enabling Semantic Search

Vector Stores

Enable semantic search by storing and querying text embeddings.

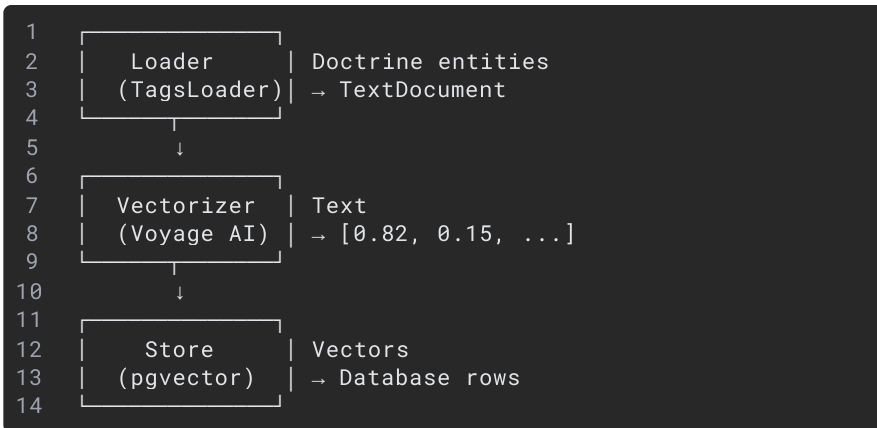
Vector Stores Enable Semantic Similarity Search

- SQL `LIKE` matches characters — misses synonyms and semantics
- Embeddings convert text to high-dimensional number arrays
- Similar meaning → numerically close vectors
- pgvector (a Postgres extension) queries by vector distance

```
1 "Docker"      → [0.82, 0.15, 0.91, ...]
2 "Container"   → [0.79, 0.18, 0.88, ...]
3
4 Distance("Docker", "Container") = 0.05 ✓
5 Distance("Docker", "Cat")       = 0.89 ✗
6
7 Query: "containerization" → [0.80, ...]
8 Result: "Docker" (distance: 0.05) ✓
```

The Indexing Pipeline Transforms Data

- **Loader:** Reads your entities
- **Vectorizer:** Creates embeddings
- **Store:** Persists vectors
- Run once, query many times



Vector Stores Persist Embeddings in PostgreSQL

- Uses pgvector extension
- Named store configurations per entity type
- `setup_options.vector_size` must match the embedding model (1024 for Voyage)
- `distance: cosine` for semantic similarity

```
1  ai:
2      store:
3          postgres:
4              posts:
5                  dbal_connection: 'doctrine.dbal...'
6                  setup_options:
7                      vector_size: 1024
8                  distance: cosine
9              tags:
10                 dbal_connection: 'doctrine.dbal...'
11                 setup_options:
12                     vector_size: 1024
13                 distance: cosine
```


Vectorizers Convert Text to Embeddings

- Platform: Voyage AI
- Model: voyage-3.5-lite
- 1024-dimensional vectors
- Captures semantic meaning

```
1  ai:  
2    vectorizer:  
3      voyage:  
4        platform: 'ai.platform.voyage'  
5        model: 'voyage-3.5-lite'
```

Document Loaders Transform Entities to Documents

- Implement `LoaderInterface`
- `load()` yields documents
- `TextDocument` with ID + content
- Metadata for filtering

```
1 class TagsLoader implements LoaderInterface
2 {
3     public function load(?string $source = null): iterable
4     {
5         foreach ($this->repo->findAll() as $tag) {
6             $metadata = new Metadata([]);
7             $metadata->setText($tag->getName());
8             yield new TextDocument(
9                 id: $tag->getId(),
10                content: $tag->getName(),
11                metadata: $metadata,
12            );
13        }
14    }
15 }
```



`Metadata::setText` stores the raw text alongside the vector, enabling **hybrid querying** on PostgreSQL — combining vector similarity with a full-text search.

Indexers Connect Loaders, Vectorizers, and Stores

- Named indexer configurations
- References loader service
- References vectorizer
- References target store

```
ai:
  indexer:
    tags:
      loader: 'App\Ai\Loader\TagsLoader'
      vectorizer: 'ai.vectorizer.voyage'
      store: 'ai.store.postgres.tags'
```

Console Commands Initialize and Populate Stores

- `ai:store:setup` creates tables
- `ai:store:index` populates data
- Run after schema changes
- Re-index when content changes

```
# Create vector store tables
php bin/console ai:store:setup

# Index all tags
php bin/console ai:store:index tags

# Index all posts
php bin/console ai:store:index posts
```

SimilaritySearch Must Be Wired Manually in services.yaml

- Not autowired — shared interface types cause ambiguity
- Declare the service explicitly with its FQCN
- Bind `$retriever` to the posts retriever

```
1  services:
2      Symfony\AI\Agent\Bridge\SimilaritySearch\SimilaritySearch:
3          arguments:
4              $retriever: '@ai.retriever.posts'
```

Phase 5

Building Advanced RAG Tools

RAG Tools

Combine retrieval with generation for context-aware operations.

RAG: Retrieval-Augmented Generation

- Retrieval: Find relevant data
- Augmented: Add to prompt
- Generation: LLM responds
- Best of both worlds!

```
1  Question: "Summarize Docker posts"
2      ↓
3
4  [ 1. RETRIEVE
5    Search vector store
6    → Find 3 Docker posts ]
7      ↓
8
9  [ 2. AUGMENT
10   Add posts to prompt:
11   "Given these posts: ..." ]
12      ↓
13
14 [ 3. GENERATE
15   LLM creates summary
16   from actual content ]
17
18
19
```


RAG Prevents Hallucinations

- LLM only uses provided data
- Answers grounded in facts
- No invented information
- Traceable to source

```
1 Without RAG:
2
3 [ Q: "What's in my posts?" |
4 [ A: "I'll make something up" | ← ❌
5
6
7 With RAG:
8
9 [ Q: "What's in my posts?" |
10 [Retrieves actual posts] |
11
12 [ A: "Based on your content,
13 your posts discuss..." | ← ✓
14
15
```

Retrievers Return Scored Documents

- Query with natural language
- Returns ranked matches
- Score: 0.0 (exact) → higher = less similar
- Filter by threshold

```
1  $docs = $retriever->retrieve("Docker tips");
2
3  // Returns:
4  [
5      Document {
6          id: "post-42",
7          content: "Docker performance...",
8          score: 0.11 // Very similar!
9      },
10     Document {
11         id: "post-17",
12         content: "Container basics...",
13         score: 0.28 // Related
14     },
15     Document {
16         id: "post-5",
17         content: "PHP coding...",
18         score: 0.85 // Not relevant
19     }
20 ]
```

Retrievers Find Semantically Similar Documents

- Inject `RetrieverInterface`
- Use `#[Target]` for named service
- `retrieve()` returns ranked docs
- Score indicates similarity

```
1 public function __construct(  
2     #[Target('tags')]  
3     private readonly RetrieverInterface $retriever,  
4 ) {}  
5  
6 $docs = $this->retriever->retrieve($content);  
7 // Returns documents ranked by similarity
```

The SimilarityPostTagger Uses RAG Pattern

- Finds posts without tags
- Retrieves similar tags by content
- Filters by similarity score
- Assigns matching tags

```
1  foreach ($postsWithoutTags as $post) {  
2      $tagDocs = $this->retriever  
3          ->retrieve($post->getContent());  
4  
5      foreach ($tagDocs as $doc) {  
6          if ($doc->score >= 0.2) {  
7              $tag = $this->tagRepo->find($doc->id);  
8              $post->addTag($tag);  
9          }  
10     }  
11 }
```

Tools Can Call the LLM Directly for Generation

- Inject `PlatformInterface`
- Create `MessageBag` with prompt
- Invoke with model constant
- Parse response content

```
1 public function __construct(  
2     #[Autowire('@ai.platform.anthropic')]  
3     private readonly PlatformInterface $platform,  
4 ) {}  
5  
6 $response = $this->platform->invoke(  
7     'claude-haiku-4-5-20251001',  
8     new MessageBag(Message::ofUser($prompt))  
9 );  
10 $result = $response->getResult()->getContent();
```

Key Takeaways

LiveComponents enable reactive PHP UIs without JavaScript

Agents abstract LLM communication with configurable prompts

Tools extend agent capabilities with application logic

Vector stores enable semantic similarity search

RAG combines retrieval with generation for intelligent automation

Questions?



Repository: github.com/Tiriell/agent-workshop



Documentation: symfony.com/doc/current/ai.html